

Mixed Discrete-Continuous Optimization for Graph-Structured Models:

Applications in Phylogenetics, Potts Models, and Hidden Markov Models

M. J. Wilson

January 1, 2025

Abstract

Inference over graph-structured models—phylogenetic trees, lattice-based Potts models, hidden Markov models, and the low-density parity-check codes that share their message passing—is a coupled discrete-continuous optimization problem. The structural search space is combinatorial, while scoring each candidate structure requires continuously optimizing its parameters. We present a framework that strictly decouples differentiable continuous optimization from discrete structural search, so that one optimizer serves every model class and a structural move constructs a new objective rather than taking a step inside an existing one. Framing structural search as a Markov decision process admits learned proposal policies, a critic and a planner over exact, approximate, bounded or learned surrogate objectives. We validate the engine against exact oracles—brute-force marginalization, path enumeration, and exhaustive topology enumeration—and report realized interval coverage rather than asserting it. Every comparison between methods is run at an equal budget of objective evaluations over shared seeds with a paired test, and the negative results are reported beside the positive ones: restarts are not beaten on the continuous surfaces measured, a learned tree policy ties hill climbing on the one fixture that separates them, and no site reaches a two-fold speedup from process parallelism on a four-core host. Formulations, algorithms, and the properties each is validated against are given in the companion textbook; this paper reports results.

Contents

1	Introduction	2
1.1	Scope	2
2	Methods	2
2.1	Continuous optimization	2
2.2	Differentiable likelihoods and energies	3
2.3	Discrete search and its surrogates	3
2.4	Structural search as a Markov decision process	3
2.5	Budgets and the ledger	3
3	Results	4
3.1	Simulation against its analytic truth	4
3.2	Exact inference across backends	4
3.3	Parameter recovery and interval coverage	4
3.4	Discrete search against exhaustive enumeration	5
3.5	The reward an agent can afford	5

3.6	Learned proposals against hill climbing	5
3.7	Budgeted comparisons	13
3.8	Hardware scaling	13
4	Discussion	15
5	Conclusions	15

1 Introduction

The fundamental difficulty in high-dimensional probabilistic inference is the combinatorial size of the structural search space. In phylogenetics, the number of unrooted binary topologies over n taxa is $(2n - 5)!!$ [7]. In statistical physics, the ground state of an N -dimensional Potts model requires traversing an exponentially large configuration space [4]. Decoding a hidden Markov model poses the same coupled problem on a chain [3], and decoding a low-density parity-check code poses it on the sparse bipartite graph of a code’s parity checks, where the sum-product algorithm was rediscovered as belief propagation [2]. The four are one marginalization over four structures, and two derived instances sit between them: a coupled spatio-sequential model, a Potts prior over class labels gating one hidden chain per class, and the Gaussian mixture, a hidden Markov model with the chain removed.

Classical frameworks rely on fixed, hand-designed heuristics—nearest-neighbour interchange for trees, Swendsen–Wang cluster updates for lattices—applied greedily or under simulated annealing. This paper builds those baselines first, pins each to an oracle, and only then asks whether a learned component beats them: a policy over the same neighbourhoods trained by the score-function estimator, an actor–critic and proximal policy optimization, a planner with the policy as prior and the critic as leaf value, certified bounds and learned surrogates that rank a neighbourhood before it is fitted, and a continuous relaxation of the discrete configurations. The continuous parameters are fitted by reverse-mode automatic differentiation throughout.

1.1 Scope

This paper reports what has been built and measured. The formulations it rests on—the substitution model, the Potts energy, the hidden Markov recursions, the discrete move sets, the decision process, the bounds, and the mathematical properties each algorithm is refereed by—are stated in the companion textbook rather than repeated here. Three things are outside it and are stated as such rather than left implicit. The low-density parity-check class has its simulator, decoder and oracles and no move set yet, so it appears in Section 3.2 and not in the search results. No comparison against IQ-TREE 2 or RAxML-NG has been run: the tools are not in the environment, and installing them is a ticket of its own (#126). And no accelerator exists to measure on, so hardware scaling is reported for the CPU alone.

2 Methods

The methodology rests on one division: continuous parameters are optimized by gradients on a fixed structure, while discrete structural moves are proposed by an external agent and construct an entirely new differentiable objective. A move changes what the parameter vector means and how long it is, so it cannot be a step inside a fit over a fixed-length vector.

2.1 Continuous optimization

An objective is a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with an injective map ϕ to the constrained parameters the model is stated in. The same quasi-Newton optimizer

[5] therefore minimizes negative log-likelihoods of phylogenetic alignments, free energies of Potts models, sequence log-likelihoods of hidden Markov models, the mixture likelihood and three closed-form test functions without knowing which it has. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method, at any fit whatever produced it—an expectation–maximization fit inverts the same constraint map to be given the same interval—and are refused where the information is singular. A Hamiltonian sampler over the same interface gives an interval as a posterior quantile instead, with a warm-up that adapts its step and mass matrix and a parallel-tempering ladder chosen from its measured exchange acceptance.

2.2 Differentiable likelihoods and energies

Partial likelihoods are evaluated by a post-order recursion, with branch lengths τ and the rate matrix $\mathbf{Q}$ kept inside the automatic differentiation graph, so gradients are exact rather than numerical [1, 6]. Rescaling against underflow is part of that graph. The Potts energy is normalized by a transfer matrix on a chain and by belief propagation on a lattice; the hidden Markov evidence by the forward recursion, with forward–backward exposed as an evaluator; and all of them by one sum-product and max-product implementation over a factor graph that also expresses the coupled model, pinned to each specialized evaluator and slower than it, so the specialized evaluators stay.

2.3 Discrete search and its surrogates

Hill climbing over nearest-neighbour interchange or subtree prune-and-regraft fits the continuous parameters of every candidate, counts its budget in candidates scored, and scores each topology once. A neighbour starts its fit from its parent’s branch lengths matched by leaf split, and a neighbourhood may be ranked by one cached evaluation, by a certified analytic bound, or by a learned predictor before only the top candidates are fitted; the accepted move is always fitted in full. Large parsimony runs the same climb on the Fitch or Sankoff score. Potts ground states are exact by minimum cut where the couplings are submodular, within a proved factor by alpha expansion above two states, and bounded by a semidefinite relaxation where the couplings are negative; samplers, annealing and parallel tempering run over one temperature schedule, and a Gibbs sampler and annealer over any factor graph.

2.4 Structural search as a Markov decision process

Discrete graph traversal is formulated as a Markov decision process [8]: the **state** is the current structure, its optimized continuous parameters, and observation summaries; an **action** is a structural perturbation from a classical neighbourhood; the **reward** is the improvement in the objective. Every learning claim is pinned to an oracle before it is measured: on an environment small enough to enumerate, the expected return, its gradient, the action values and the optimal value are exact. On that footing the score-function estimator with a baseline, an actor–critic with a fitted critic, proximal policy optimization with generalized advantage estimation and an optional epsilon-greedy behaviour policy, and a planner running PUCT search with the policy as prior and the critic as leaf value, trained by expert iteration, are compared at matched budgets. A Gumbel-softmax relaxation of Potts configurations and hidden state paths is the differentiable alternative, admitted where its optimum is enumerable so the claim is falsifiable.

2.5 Budgets and the ledger

Every comparison between methods spends an equal budget in one declared unit—candidate fits, sweeps, or objective evaluations—over shared seeds, derives the restart baseline’s count from that budget, refuses a method that spends past it, and reports an exact McNemar test on

the paired hits. The measured comparisons are recorded in an experiment ledger, one file per experiment against its commit, and cited here by number.

3 Results

Every figure below is rendered from the code it reports on and carries a generated caption naming the seed, sizes, and model that produced it.

3.1 Simulation against its analytic truth

$$(A_{0.1}, B_{0.25}, (C_{0.15}, D_{0.4})\alpha_{0.05})\rho$$

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 1: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200_000 sites total), showing the topology with branch lengths – rho the root, Greek letters its internal ancestors – and the first 10 of 200_000 simulated sites.

3.2 Exact inference across backends

The pruning backends are held to brute-force marginalization (Figure 2). The other evaluators are held the same way, and the parity-check decoder is the newest: on a cycle-free 22-bit code its posteriors equal the tree-schedule sum-product and the enumeration over all 32,768 codewords to 1.9×10^{-13} in log-odds, min-sum returns the enumerated maximum-likelihood codeword, and on six loopy codes it reaches the general damped flooding’s fixed point within 4.6×10^{-11} . At 19,998 bits, where no enumeration reaches, the (3,6) code resolves every erasure at an erasure probability of 0.42 on three seeds and leaves 25 to 30% of its bits erased at 0.44, bracketing the density-evolution threshold 0.4294.

3.3 Parameter recovery and interval coverage

Recovery is the acceptance test for a fit, and a Wald interval from the observed information is an *asymptotic* claim. The figures below are the evidence for it rather than an assertion of it: the first shows each fitted parameter of the Potts chain and the hidden Markov model against the value that generated the data, the second the realized coverage against the amount of data each fit sees, and the third and fourth the phylogenetic instance, branch lengths and the general

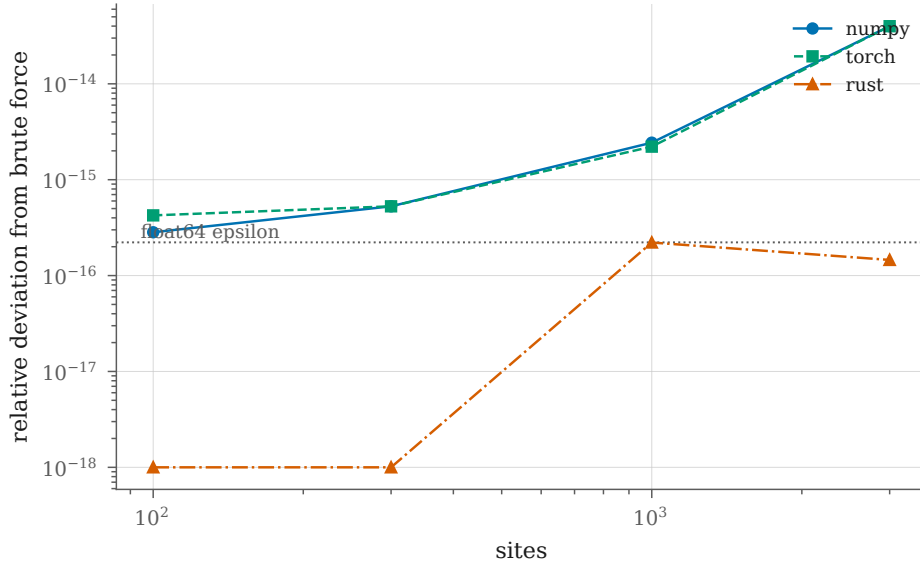


Figure 2: Every pruning backend against the oracle that owes it nothing. Brute-force marginalization sums over all ancestral assignments, so it shares no code with the recursion it checks, and its cost is what keeps the fixture small. Fixture: 4-state Jukes-Cantor on the 3-child-root tree, seed 20260902. The deviation is relative because the log-likelihood is a sum over sites: an absolute bound fixed at one site count does not transfer to another, which is why the x-axis spans a factor of 30. Worst deviation across every backend and size: $3.99\text{e-}14$, against a float64 epsilon of $2.22\text{e-}16$.

time-reversible model, where two directions of the parameter space are gauges and are fitted as such.

3.4 Discrete search against exhaustive enumeration

Below the size at which every topology can be enumerated, enumeration is the only independent statement available about whether a search found the best tree or merely a good one. The first figure is the climb and the landscape it climbed; the second the tree it selected beside the best one it rejected.

3.5 The reward an agent can afford

Fitting branch lengths per candidate costs a full optimization where scoring at fixed parameters costs one pruning pass, and only the second makes an episode affordable. The substitution is validated rather than assumed: the two surfaces agree on the best topology and correlate at 0.9568 on the six-taxon fixture, and the agreement holds across a 50-fold range of the fixed branch length.

3.6 Learned proposals against hill climbing

Where the landscape can be enumerated, learning is refereed by the exact expected return. On the Potts chain the reward decomposes into the two features the policy scores, which puts hill climbing inside the policy class: the learned policy reaches the enumerated optimum from 86.6% of the 81 starts against greedy’s 80.2%, in 8 of 8 training seeds (ledger entry 003); at the same budget the actor-critic reaches it from 88.9% and proximal policy optimization from 96.3%, and at a quarter of the budget 87.7% against the score-function estimator’s 32.1%. A planner with a

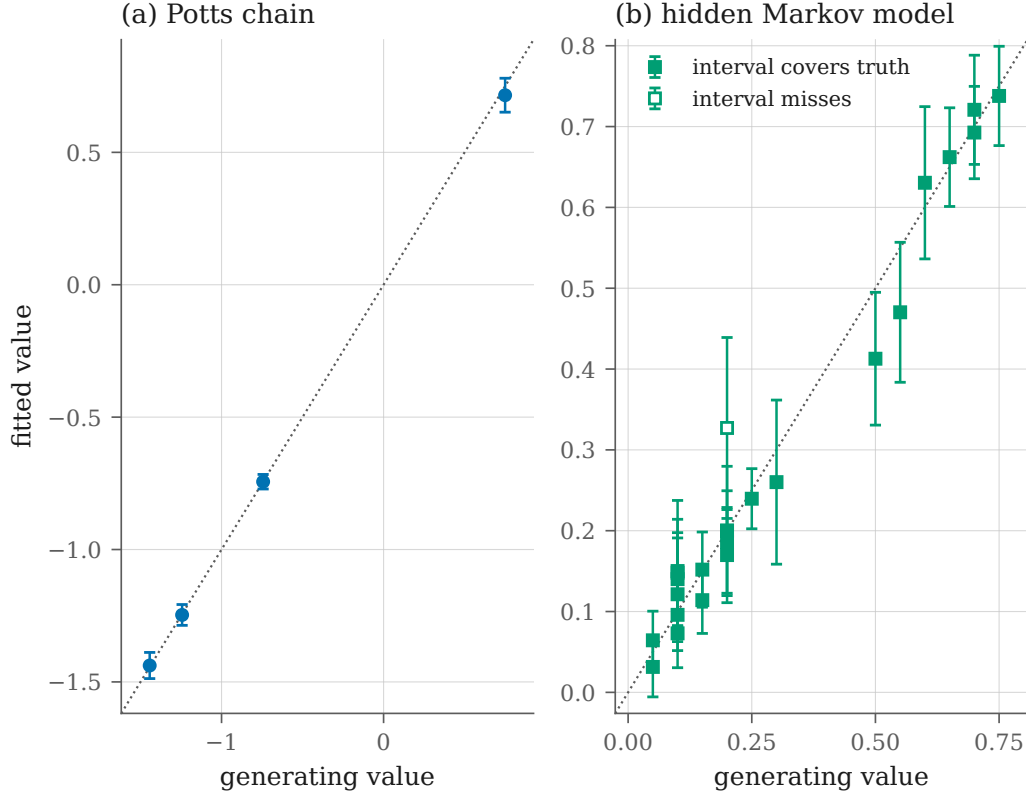


Figure 3: Parameter recovery for the two reference instances of the optimization interface, fitted by the same model-agnostic optimizer. Bars are 95 percent Wald intervals from the observed information, propagated to the reported parameters by the delta method; the dotted line is $y = x$. (a) A 3-state Potts chain, 400 chains of length 12, seed 20260902: coupling and gauge-fixed field, 4 of 4 intervals covering (1.00). (b) A 3-state, 4-symbol hidden Markov model, 600 sequences of length 15, seed 20260903: initial, transition and emission probabilities after aligning the hidden-state permutation the likelihood is invariant to, 23 of 24 intervals covering (0.96). Coverage on a single dataset is a draw, not a rate: the nominal 0.95 is checked over independent replicates by the fitting regression suite, and against sample size in the companion coverage figure.

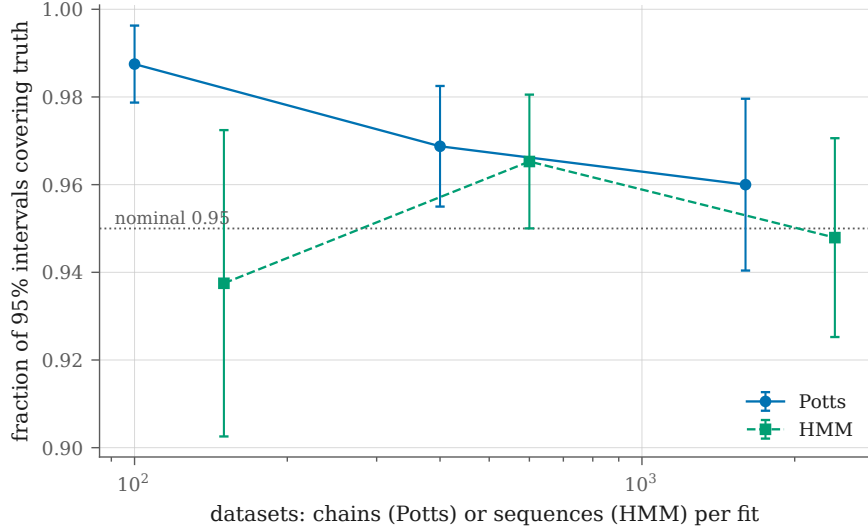


Figure 4: Realized coverage of the 95 percent Wald intervals built from the observed information, against the amount of data each fit sees. Bars are binomial standard errors on the realized fraction. Both instances converge on the nominal rate and neither sits on it at the smallest sample, which is what an asymptotic approximation does and a wrong formula does not. They approach from opposite sides. The Potts chain over-covers and comes down: $158/160 = 0.988$ at 100 chains, $96/100 = 0.960$ at 1600. The hidden Markov model under-covers and comes up: $45/48 = 0.938$ at 150 sequences, $91/96 = 0.948$ at 2400. The under-coverage has two identifiable sources absent from the Potts chain – some emission probabilities are fitted near zero, where a Wald interval on the log scale is a poor approximation, and aligning the hidden-state permutation to the truth is a post-selection step that costs a little coverage. Seeds are fixed, so every point is reproducible rather than redrawn. At the smallest hidden Markov size 6 of 18 fits reached the boundary of the parameter space – an emission probability estimated at zero – where the observed information is singular and there is no interval to report. Those fits are excluded from the counts above and stated here rather than dropped silently, since excluding them unannounced would select for the well-behaved samples.

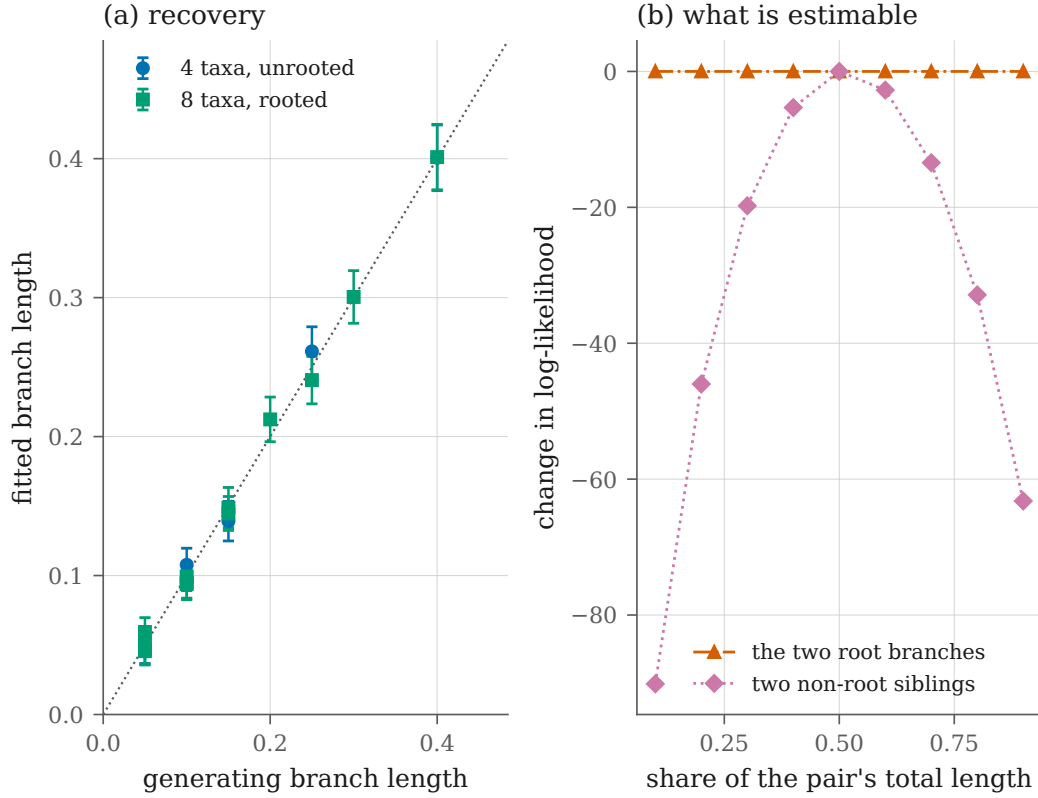


Figure 5: Branch-length fitting by autodiff, using the same model-agnostic optimizer as the Potts and hidden Markov instances. (a) Every estimable branch length against the value that generated the alignment, with 95 percent Wald intervals from the observed information; markers are open where an interval misses. Fixtures: 3-child root, seed 20260903, and 2-child root, seed 20260904, both scored at 5000 sites. (b) Why some branch lengths are not estimable at all. Moving mass between the two branches below a rooted binary tree’s root, at fixed sum, changes the log-likelihood not at all: the log-likelihood is bit-identical across a 9 to 1 range. The same move between two non-root siblings changes it by up to 90.1. This is the pulley principle: under a reversible model the likelihood does not depend on where the root sits along the branch it subdivides. Only the pair’s sum is estimable, so the objective fits it as one parameter; fitting both would leave the observed information singular and every interval in panel (a) undefined.

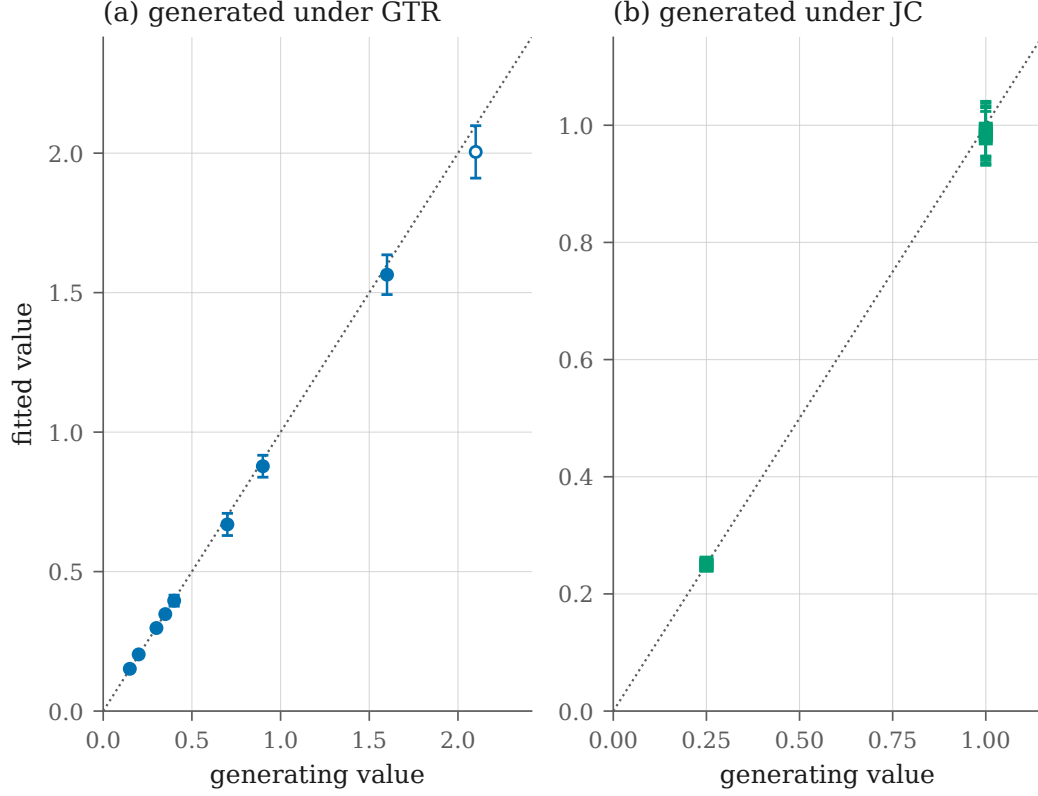


Figure 6: Fitting the general time-reversible model: five free exchangeabilities (the sixth is pinned at 1) and four stationary frequencies, alongside the branch lengths, by the same model-agnostic optimizer used for the Potts and hidden Markov instances. Bars are 95 percent Wald intervals from the observed information; the dotted line is $y = x$; markers are open where an interval misses. Topology: the 2-child-root fixture, seed 20260904, 20000 sites. (a) Data simulated under an asymmetric truth, 8 of 9 intervals covering. (b) Data simulated under Jukes-Cantor, where every exchangeability is 1 and every frequency $1/4$: 9 of 9 covering. The second panel is the half that is easy to forget. A model flexible enough to fit panel (a) is only useful if it also declines to invent structure that is not there, and Jukes-Cantor is exactly the point of the parameter space where the general model could most easily do so. Coverage on a single dataset is a draw and not a rate: the nominal 0.95 is checked over independent replicates by the release-gated regression suite.

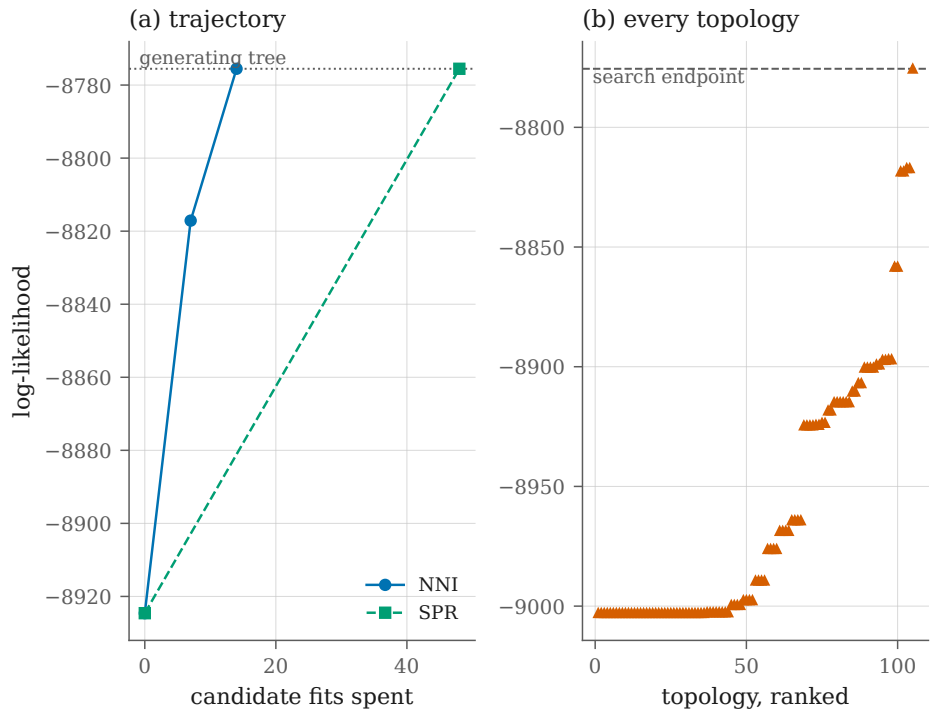


Figure 7: Hill climbing over tree topologies, scoring every candidate with the same model-agnostic optimizer used for the Potts and hidden Markov instances. Fixture: 4-state Jukes-Cantor, 1500 sites, seed 20260905. (a) Log-likelihood against candidate fits spent, from a randomly drawn starting topology; the dotted line is the generating tree’s own score. (b) All 105 unrooted topologies on this leaf set, scored and ranked, with the search’s endpoint dashed. Below 8 taxa this enumeration is affordable, which is what makes the phrase found the best tree a checkable claim rather than an assertion. The optimum leads the runner-up by 41.6 log units here, so this problem does not yet separate the two move sets: both reach it, NNI in fewer fits.

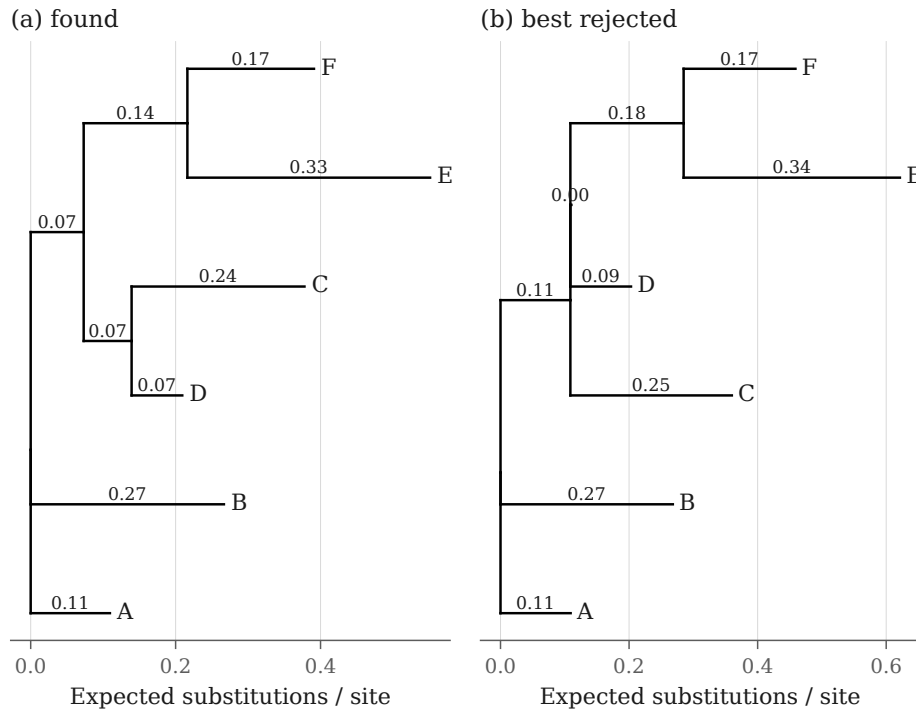


Figure 8: The topology hill climbing selected, beside the highest-scoring one it rejected, each with its own branch lengths fitted. Fixture: 4-state Jukes-Cantor over 1500 sites, seed 20260905. (a) scores -8775.5 and is the generating topology; (b) scores -8817.1, a difference of 41.6 log units. They differ by one split: C D are a group in one and not in the other. On this fixture the search recovers the generating tree, so a drawing of found against true would show the same tree twice; the runner-up is what the likelihood actually had to discriminate against. Note its shortest internal branch, fitted at 0.000: a wrong grouping is generally only supportable by collapsing the edge that would create it.

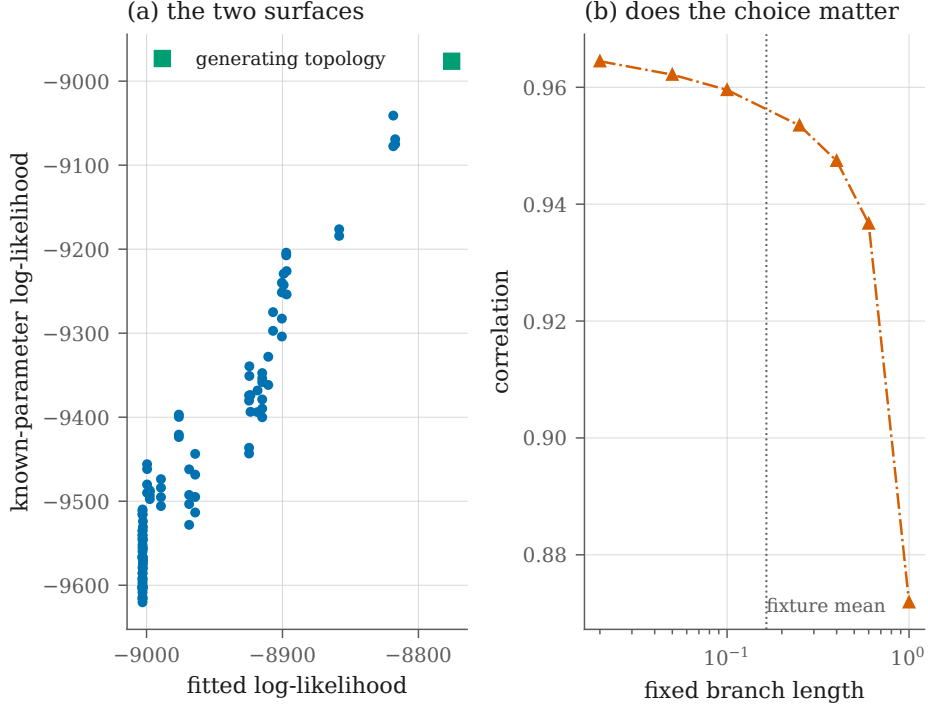


Figure 9: The reward an agent can afford, against the one it stands in for. Every one of the 105 unrooted topologies on this leaf set, scored twice: at fixed known parameters, and at parameters maximized per topology by the same optimizer the Potts and hidden Markov instances use. Fixture: 4-state Jukes-Cantor, 1500 sites, seed 20260905; the fixed branch length is the generating tree’s mean, 0.1644. (a) The two surfaces, correlation 0.9568; both score the generating topology highest, so the cheap reward and the expensive one agree on the answer while disagreeing in detail. The correlation is linear rather than rank-based because the fitted surface does not totally order topologies: several of its optima agree to within the optimizer’s own convergence, so their relative order is not a property of the model. (b) The cheap surface has one free parameter, and the conclusion does not rest on it: across a 50-fold range of fixed branch length the two surfaces pick the same best topology in 7 of 7 cases, with the correlation never falling below 0.8719. The substitution is what makes training affordable at all: the fitted reward runs a full optimization per candidate, the known one a single pruning pass.

trained prior and critic reaches it from 92.6% of starts at 8.3 successor evaluations per episode against greedy’s 80.2% at 48, and matches greedy at 6.3.

Figure 10 poses the phylogenetic comparison on a topology landscape where hill climbing demonstrably fails, which is the condition that makes the comparison askable at all: on a landscape the baseline already solves from every start, a policy can be neither better nor worse than it. The answer is a tie. Every algorithm—the score-function estimator, the actor-critic, proximal policy optimization on and off policy—lands on greedy’s 0.48 within the 0.014 standard deviation over seeds, because the environment exposes one feature, the improvement a move buys, so the policy is an inverse temperature and no algorithm can learn what that class cannot express. Random-restart hill climbing reaches the enumerated maximum from every start at the same 60-decision budget, so the baseline the next comparison has to beat is 1.000 and a fixture that separates anything from restarts is what it needs. Given something to learn the policy learns: with seven columns per move—the improvement, the parsimony change, the pattern support of the split broken and of the split made, and the two exchanged subtree sizes—it reaches the maximum from 0.796 of episodes at the same budget, ahead of greedy on 16 of 16 seeds ($p = 3.05 \times 10^{-5}$, ledger entry 005), and is not yet measured against restarts.

3.7 Budgeted comparisons

Which search wins is a property of the instance, and every comparison below is at an equal budget over shared seeds. On the planted Viana–Bray spin glass at 60 sites, degree 4 and frustration 0.2, at 400 sweeps per method over 12 instances, parallel tempering reaches the best energy any method found on 12 of 12, annealing on 10 of 12 with a mean gap of 0.17, and restarts of descent on 4 of 12 with a mean gap of 0.75; every method beats the planted energy. On the five-component Gaussian mixture at 3,000 likelihood evaluations per start over 40 shared starts, restarts of expectation–maximization reach the best-known optimum from 7 starts, parallel tempering from 4 ($p = 0.549$ against restarts) and annealing from 1 ($p = 0.031$), with mean gaps of 2.6, 4.2 and 8.0 nats (ledger entry 004). On Rastrigin, held to eight fits, neither multi-start nor annealing reaches the global minimum from any of 10 starts. Restarts are therefore not beaten on the continuous surfaces measured and are beaten on the frustrated lattice, the one instance class past enumeration the roadmap needs. On an antiferromagnetic chain whose optimum needs coordinated flips the deterministic relaxation reaches it from 18 of 40 seeds against greedy’s 5 ($p = 0.00098$), and every sampled variant of it ties at 11 ($p = 0.18$). At the Potts critical coupling, single-site sampling slows by 3.2 times between lattice extents 8 and 24 while the cluster moves slow by 1.9 (ledger entry 001). The comparison against classical phylogenetic software is ticketed (#126) and not run.

3.8 Hardware scaling

No accelerator exists in the environment, so what is measured is the CPU. The Rust pruning backend runs at 2.5 times the NumPy oracle at 200 taxa by 11,000 sites, with peak memory 22.6 MB, and the minimum-cut kernel at 26 to 32 times the reference as a kernel and 6.3 to 10.7 times as a caller sees it, the difference being the Python energy evaluation and not the boundary copy. Process parallelism over independent tasks—the multi-start fit, the budgeted comparison, the bootstrap—was measured on a four-core host at 1, 2 and 4 workers with the intra-op thread count at 1 and at the default: no site reaches a two-fold speedup at 4 workers, the pool’s spawn and import costing 1.81 s, more than the multi-start fit does in total, while the serial fit at the default thread count runs 2.87 times the fit at one thread because the tensor library already parallelizes over sites. All three sites stay serial by recommendation, with the argument kept for a larger tier or machine. Table 1 settles the memory requirement: the roadmap bounds the working set to $O(n \times L \times k)$ inside 16 GB, the simulator retains every node’s states and pruning

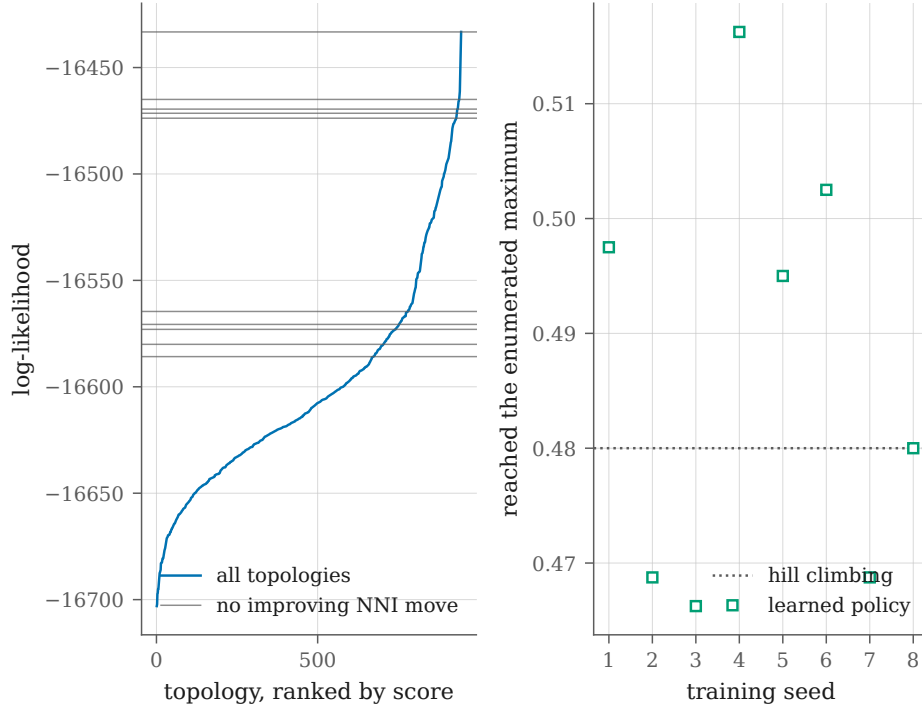


Figure 10: A learned policy against hill climbing on a landscape where hill climbing fails, and why the answer is a null result. Fixture: 945 unrooted topologies on 7 taxa, 2000 sites, 4-state Jukes-Cantor, seed 20.260_907, scored at the generating tree’s mean branch length. (a) Every topology, ranked, with a rule at each of the 10 states no single NNI move improves. The best of them is the generating topology; the rest are traps. (b) The fraction of episodes reaching that maximum, from 50 shared starting topologies at 30 moves each – greedy once per start, the policy 16 times per start so its rate is an average and not a best-of, which makes the budgets comparable. Over 8 independent training runs of 640 episodes each, the policy reaches the maximum on 0.487 of episodes against greedy’s 0.480: a difference of +0.0069 with a standard deviation of 0.0186 across seeds, and 4 of 8 seeds ahead. That is not a difference. It is not that nothing was learned: an untrained policy, uniform over the same moves, reaches the maximum on 0.018 of episodes, so training moves the policy across most of the distance from chance to the baseline and then stops there. Panel (a) says why it stops there: an episode ends when no move improves, so every run terminates at one of those rules and the task is which trap to enter rather than how to leave it, while a policy scoring moves by this environment’s single feature – the improvement a move buys – is hill climbing up to a temperature.

retains a partial likelihood only for nodes whose parent has not yet consumed it, and both are reported on a caterpillar, the deepest tree on its leaves, so the bound is tight rather than loose.

Taxa	Sites	Simulate (MB)	Evaluate (MB)	Total (MB)
20	2000	0.624	2.43	3.06
20	11_000	3.43	13.4	16.8
100	11_000	17.5	69.7	87.2
1000	11_000	176	703	879

Table 1: Memory held while simulating a Jukes-Cantor alignment and while evaluating its likelihood by pruning, at 4 states, across the declared scale. The simulator retains every node’s states and pruning retains one partial likelihood per open node, so both are computed from the arrays’ own shapes and are exact rather than sampled; the regression suite pins each figure against the allocator’s count. The last row is the declared maximum of 1000 taxa and 11_000 sites, which sits a factor of 20 inside the 16 GB unified-memory requirement. The topology is a caterpillar at every size, the deepest tree on its leaves and so the worst case: a balanced tree of the same taxon count costs strictly less. Timings are reported in the benchmark suite instead, because a wall clock ranks the machine rather than the code.

4 Discussion

Three findings recur across the problem classes, and each is a statement about the instance rather than about the algorithms. First, most fixtures are too easy to separate methods: the six-taxon tree is solved from every start, the Potts chain’s optimum is a constant, the triangular antiferromagnet is reached by restarts of descent as often as by annealing, and it took a planted glass past enumeration and a seven-taxon tree with short internal branches to make a comparison askable. The budgeted comparison is therefore run before any claim is made, and a fixture that fails to separate methods is recorded as such. Second, where a learned component loses or ties, the cause is named and measured rather than argued: the tree policy ties greedy because its feature set is one number, and the relaxation’s sampled variants tie greedy because the sampling, not the relaxation, is what costs. Third, the negative engineering results are as load-bearing as the positive ones: the fourth-order integrator loses to leapfrog at equal gradient evaluations, k-means++ seeding buys nothing the likelihood can use, warm starts save less than lazy scoring, and process parallelism buys nothing at the mid-size tier. Each stays as a measured decision rather than a default that moved.

What referees each method is read from the suite and typeset in the companion textbook: where an oracle is affordable at a size the method is pinned to it, and where none is, recovery of the simulated truth stands in while the oracle remains desired. The cells where that second case holds are marked there, and the oracle each would need is named—a certified branch-and-bound bound for trees past the enumerable taxon count, a boundary contraction for lattices past the transfer-matrix width, and density evolution for codes past the enumerable length.

5 Conclusions

Decoupling structural exploration from continuous fitting yields a model-agnostic optimization engine that serves phylogenetics, Potts models, hidden Markov models, the parity-check code, the coupled model and the mixture without encoding any of them. Every evaluator is pinned to an oracle that shares no code with it, every fit is judged by recovery at its nominal coverage, and every search by the enumeration that referees it where enumeration reaches. Formulating

the search as a Markov decision process gave a critic, a policy-gradient family and a planner that beat hill climbing where the landscape can be enumerated and tie it on the tree fixture, for a reason that is measured and ticketed. What remains is stated as tickets rather than as future work: the feature set that would change the tree result, the code's move set and its error rates at size, the external tools the competitiveness claim needs, and the accelerator the scaling claim needs.

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.
- [2] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [3] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.
- [4] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.
- [5] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [6] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.
- [7] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.
- [8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.